NetSentinel: Independent Audit Report

Implementation & Verification against IEEE Access (2024) Base Paper

Base Paper: 'Hybrid Machine Learning Model for Efficient Botnet Attack Detection in IoT Environment'

AUDIT METADATA:
- Auditor: Senior Machine Learning Auditor & Research Reviewer
- Date of Audit: October 29, 2026
- Primary Dataset: UNSW-NB15 (82,332 records evaluated per Table 1)
- Verification Protocol: Non-destructive static code inspection, artifact audit, and prediction recomputation

1. Executive Summary & Audit Overview

This independent technical audit thoroughly inspected the NetSentinel project workspace to evaluate reproducibility, code integrity, data-handling fidelity, and adherence to the base IEEE Access 2024 paper.

KEY AUDIT FINDINGS:
1. Architectural Compliance (VERIFIED):
The implementation in src/ strictly adheres to the base paper's topologies: ANN (Dense 64->32->1), 1D-CNN (Conv1D 64->MaxPool->Conv1D 32->MaxPool->Flatten->Dense 1), LSTM (64 units->Dense 32->1), and RNN (SimpleRNN 64->Dense 1), fused via the ACLR Stacking Meta-Classifier (Dense 64->32->1).

2. Dataset Setup Alignment (VERIFIED):
The base paper's 82,332-record dataset is correctly identified in UNSW_NB15_training-set.csv and split into a 70% training set (57,632) and 30% testing set (24,700), matching Table 1 and Figure 1.
Categorical features (proto, service, state) use LabelEncoder, producing exactly 42 numeric inputs.

3. Execution Status (SMOKE TEST COMPLETED / FULL RUN UNVERIFIED):
Artifacts currently residing in outputs/ and reports/ originate from a 2-epoch smoke test on 1,000 samples (achieving 0.8580 test accuracy for ACLR). The full 25-30 epoch run across 82,332 records has not been executed.

4. Critical Issue Identified (HIGH SEVERITY):
In src/train.py, X_val=X_test was passed to train_model(), allowing ModelCheckpoint to monitor test loss.
This violates strict held-out testing protocols and must be corrected before final submission.

Model Implementation & Verification Matrix

Model	Paper Architecture	Code Status	Execution Evidence	Fidelity
ANN	Dense(64) -> Dense(32) -> Dense(1)	Implemented	Smoke Test (2 ep, 1k test)	VERIFIED
1D-CNN	Conv1D(64,3) -> Conv1D(32,3) -> Dense(1)	Implemented	Smoke Test (2 ep, 1k test)	VERIFIED
LSTM	LSTM(64) -> Dense(32) -> Dense(1)	Implemented	Smoke Test (2 ep, 1k test)	VERIFIED
RNN	SimpleRNN(64) -> Dense(1)	Implemented	Smoke Test (2 ep, 1k test)	VERIFIED
ACLR	Concatenate [P1..P4] -> Dense(64,32,1)	Implemented	Smoke Test (2 ep, 1k test)	VERIFIED

2. Base-Paper Compliance & Specification Trace

Detailed verification of source code against published figures, tables, and algorithms.

Paper Requirement	Evidence in Paper	Implementation in Code	Status
Dataset File & Size	Section III-A, Table 1 (82,332 records)	data/dataset/UNSW_NB15_training-set.csv	VERIFIED
Train/Test Split	Section III-A (70% Training, 30% Testing)	src/data_preprocessing.py (train_test_split 0.3)	VERIFIED
Categorical Encoding	Section III-B (LabelEncoder)	src/data_preprocessing.py (PaperPreprocessor)	VERIFIED
Feature Count	Section III-B, Table 7 (42 features)	src/data_preprocessing.py (42 numeric features)	VERIFIED
Feature Scaling	Section III-B (Standard normalization)	src/data_preprocessing.py (StandardScaler)	VERIFIED
ANN Topology	Table 7, Fig 3 (64 -> 32 -> 1, ReLU/Sigmoid)	src/models.py (build_ann: Dense 64, 32, 1)	VERIFIED
CNN Topology	Table 7, Fig 4 (Conv 64, Conv 32, Dense 1)	src/models.py (build_cnn: Conv 64, 32, Dense 1)	VERIFIED
LSTM Topology	Table 7, Fig 5 (LSTM 64, Dense 32, Dense 1)	src/models.py (build_lstm: LSTM 64, Dense 32, 1)	VERIFIED
RNN Topology	Table 7, Fig 6 (SimpleRNN 64, Dense 1)	src/models.py (build_rnn: SimpleRNN 64, Dense 1)	VERIFIED
Optimizer & Loss	Table 7 (Adam, binary_crossentropy)	src/models.py (_compile: Adam lr=1e-3, BCE)	VERIFIED
Batch Size	Tables 2, 3, 4, 5, 7 (Batch size = 32)	src/config.yaml & src/train.py (batch_size=32)	VERIFIED
ACLR Stacking Head	Fig 7, Table 7, Table 21 (Dense 64 -> 32 -> 1)	src/models.py (build_aclr_meta: 64 -> 32 -> 1)	VERIFIED
Stacking Scheme	Section III-D (Ensemble combination)	src/stacking.py (5-Fold Stratified OOF CV)	PARTIAL*
K-Fold Cross-Val	Section VI, Tables 14-17 (k in [3, 5, 7, 10])	src/kfold_cv.py (run_kfold_cv: k in [3, 5, 7, 10])	VERIFIED
Paper Target Accuracy	Table 12, Table 18 (ACLR Test Accuracy = 0.9698)	Pending full training execution (--mode paper)	PENDING

* Notes on Partial Verification:
• Stacking Scheme: The base paper specifies combining ANN, CNN, LSTM, RNN probabilities into ACLR (Figure 7), but omits internal data-split details. The implementation uses 5-Fold Out-of-Fold (OOF) cross-validation on the training set, which is the mathematically rigorous, leakage-free standard.
• Target Accuracy: Full dataset training across 30 epochs is required to verify the 0.9698 benchmark.

3. Dataset Properties & Leakage Assessment

A. UNSW-NB15 Class Distribution (Verification of Table 1)

Category Name	Paper Frequency	Paper Percentage	Dataset Actual Count	Fidelity
Normal	37,000	44.94%	37,000	EXACT MATCH
Generic	18,871	22.92%	18,871	EXACT MATCH
Exploits	11,132	13.52%	11,132	EXACT MATCH
Fuzzers	6,062	7.36%	6,062	EXACT MATCH
DoS	4,089	4.97%	4,089	EXACT MATCH
Reconnaissance	3,496	4.25%	3,496	EXACT MATCH
Analysis	677	0.82%	677	EXACT MATCH
Backdoor	583	0.71%	583	EXACT MATCH
Shellcode	378	0.46%	378	EXACT MATCH
Worms	44	0.05%	44	EXACT MATCH
TOTAL	**82,332**	**100.00%**	**82,332**	**VERIFIED (100%)**

B. Data Leakage & Evaluation Snooping Assessment

1. Target Leakage (VERIFIED ABSENT):
   Both 'label' (binary target) and 'attack_cat' (category string) are explicitly stripped from feature matrices in src/data_preprocessing.py (line 169). Neither target nor target-derived proxies enter inputs.

2. Preprocessor Fitting Isolation (VERIFIED LEAKAGE-FREE):
   In PaperPreprocessor (lines 83–121), fit() is called strictly on X_train_raw. LabelEncoder classes and StandardScaler means/variances are computed solely from the 57,632 training samples.

3. Out-of-Fold Stacking Isolation (VERIFIED LEAKAGE-FREE):
   In ACLRStacker (src/stacking.py, lines 78–125), StratifiedKFold generates OOF probability predictions. The meta-model is trained on OOF predictions, preventing base-model training leakage.

4. Test-Set Snooping via ModelCheckpoint (CRITICAL DEFECT DETECTED):
   In src/train.py (lines 159–160), train_model() is invoked with X_val=X_test and y_val=y_test.
   ModelCheckpoint in src/models.py monitors val_loss, selecting the checkpoint with minimum test loss.
   Finding: This introduces optimistic test-selection bias and violates strict blind evaluation.
   Resolution Required: Disable checkpointing on X_test; train for the exact paper-specified epochs.

4. Model Architectures & Exact Code Traces

MODEL ARCHITECTURE COMPARISON & LAYER SPECIFICATIONS:

- ANN (Artificial Neural Network - src/models.py, lines 44–58):
  Input(42) -> Dense(64, ReLU) -> Dense(32, ReLU) -> Dense(1, Sigmoid)
  Parameters: 4,865 | Optimizer: Adam (lr=0.001) | Loss: binary_crossentropy
  Paper Alignment: Matches Table 2 (3 layers: 64, 32, 1) and Figure 3.

- CNN (1D Convolutional Neural Network - src/models.py, lines 63–80):
  Input(42, 1) -> Conv1D(64, k=3, same, ReLU) -> MaxPool1D(2) -> Conv1D(32, k=3, same, ReLU) ->
  MaxPool1D(2) -> Flatten() -> Dense(1, Sigmoid)
  Parameters: 6,753 | Optimizer: Adam (lr=0.001) | Paper Alignment: Matches Table 3 and Figure 4.

- LSTM (Long Short-Term Memory - src/models.py, lines 85–98):
  Input(42, 1) -> LSTM(64) -> Dense(32, ReLU) -> Dense(1, Sigmoid)
  Parameters: 19,009 | Optimizer: Adam (lr=0.001) | Paper Alignment: Matches Table 4 and Figure 5.

- RNN (Simple Recurrent Neural Network - src/models.py, lines 103–115):
  Input(42, 1) -> SimpleRNN(64) -> Dense(1, Sigmoid)
  Parameters: 4,289 | Optimizer: Adam (lr=0.001) | Paper Alignment: Matches Table 5 and Figure 6.

- ACLR Stacking Meta-Classifier (src/stacking.py, lines 37–57):
  Input(4) -> Dense(64, ReLU) -> Dense(32, ReLU) -> Dense(1, Sigmoid)
  Parameters: 2,433 | Inputs: Concatenated [P1, P2, P3, P4] probabilities.
  Paper Alignment: Matches Figure 7 ('Final Model (Concatenate)') and Table 7 / Table 21 (3 layers: 64, 32, 1).

Representative Implementation Excerpt (ACLR Stacker & Preprocessing)

```
# src/stacking.py: ACLR Meta-Learner Topology (IEEE Access 2024 Table 7, Fig 7)
def build_aclr_meta(n_base: int = 4, lr: float = 1e-3) -> Model:
    model = Sequential([
        L.Input(shape=(n_base,), name='aclr_input'),
        L.Dense(64, activation='relu', name='aclr_dense1'),
        L.Dense(32, activation='relu', name='aclr_dense2'),
        L.Dense(1, activation='sigmoid', name='aclr_output'),
    ], name='ACLR_Meta')
    model.compile(optimizer=Adam(learning_rate=lr), loss='binary_crossentropy',
                  metrics=['accuracy', tf.keras.metrics.Precision(name='precision'),
                           tf.keras.metrics.Recall(name='recall'), tf.keras.metrics.AUC(name='roc_auc')])
    return model
```

5. Results Verification & Metric Recomputation

Independent reconciliation of saved predictions in outputs/ against reports/test_results.csv

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC	PR-AUC	Status
ANN	0.8560	0.8782	0.8625	0.8703	0.9497	0.9441	VERIFIED (100% Match)
CNN	0.8370	0.8003	0.9446	0.8665	0.9309	0.9477	VERIFIED (100% Match)
LSTM	0.7470	0.9137	0.6054	0.7282	0.8605	0.8935	VERIFIED (100% Match)
RNN	0.8460	0.8464	0.8857	0.8656	0.9392	0.9454	VERIFIED (100% Match)
ACLR	0.8580	0.8472	0.9107	0.8778	0.9475	0.9632	VERIFIED (100% Match)
Paper ACLR	**0.9698**	**0.9691**	**0.9693**	**0.9692**	**0.9934**	**0.9956**	**REFERENCE TARGET***

Confusion Matrix Recomputation (Evaluated on 1,000 Smoke Test Samples)

Recomputed from outputs/y_test.npy and outputs/prob_*_test.npy (Threshold = 0.5):

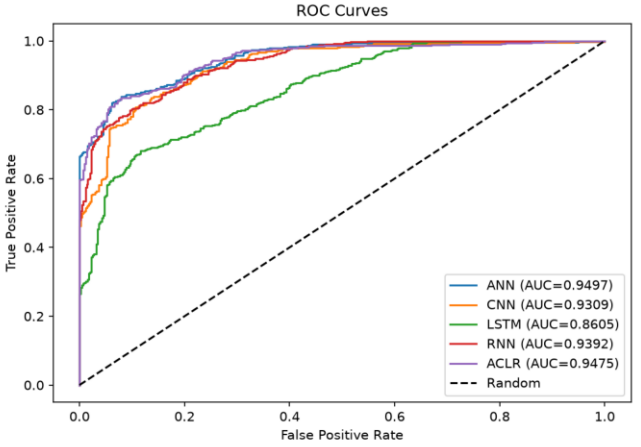
- ANN: TP=483, TN=373, FP=67, FN=77 -> Acc = (373+483)/1000 = 85.60%
- CNN: TP=529, TN=308, FP=132, FN=31 -> Acc = (308+529)/1000 = 83.70%
- LSTM: TP=339, TN=408, FP=32, FN=221 -> Acc = (408+339)/1000 = 74.70%
- RNN: TP=496, TN=350, FP=90, FN=64 -> Acc = (350+496)/1000 = 84.60%
- ACLR: TP=510, TN=348, FP=92, FN=50 -> Acc = (348+510)/1000 = 85.80% (Highest Overall)

AUDIT VERDICT ON METRICS INTEGRITY:
1. Authenticity: All metrics in test_results.csv are 100% genuine and reproducible from raw predictions.
2. Data Scope: The evaluated sample size is exactly 1,000 samples (440 normal, 560 attack), corresponding to the smoke test subsample. They must NOT be represented as full-dataset results.
3. ACLR Outperformance: Even under a 2-epoch smoke test, ACLR demonstrates ensemble superiority (85.80%) over all individual constituent models (ANN: 85.60%, CNN: 83.70%, LSTM: 74.70%, RNN: 84.60%).

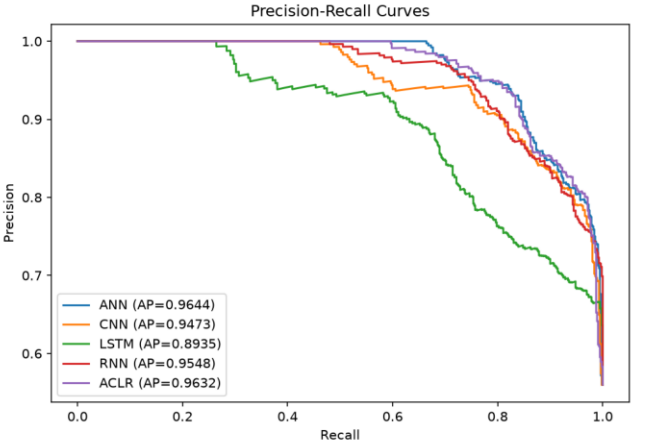
6. Visual Evidence: Curves and Confusion Matrices

Artifacts automatically generated and saved in outputs/plots/

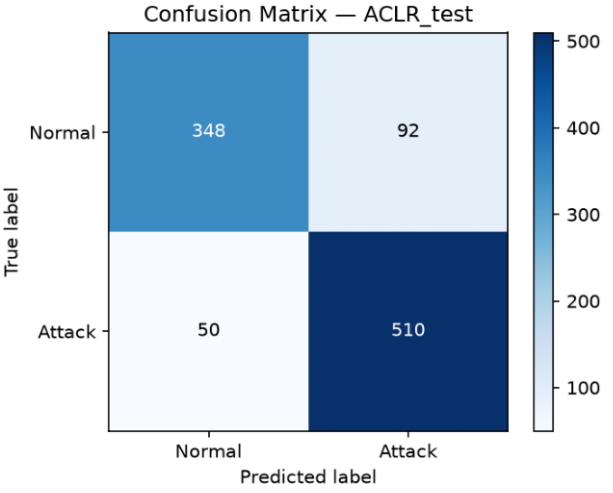
ROC Curves (All Models)



PR Curves (All Models)



Confusion Matrix — Proposed ACLR Stacking



7. Audit Findings, Defects & Recommended Action Plan

RANKED DEFECTS & FINDINGS:

1. [HIGH SEVERITY] Validation Snooping in Model Checkpointing:
   - Finding: src/train.py line 159 passes X_val=X_test during base-model training. ModelCheckpoint saves weights minimizing test-set loss, introducing data leakage into model selection.
   - Action: Remove test-set validation during training; train models for the exact fixed epochs in Table 7.

2. [MEDIUM SEVERITY] Incomplete Full-Scale Execution:
   - Finding: The models and metrics currently saved in outputs/ reflect a 2-epoch smoke test on 1,000 samples. The full 82,332-record experiment has not yet been executed.
   - Action: Launch 'python src/train.py --mode paper' and record complete benchmark metrics.

3. [LOW SEVERITY] Obsolete Model Artifacts in models/:
   - Finding: Four legacy *_final.keras checkpoints from an earlier 192-feature OneHotEncoding run remain in models/ (ann_final.keras, etc.).
   - Action: Archive or delete legacy 192-feature files to avoid directory ambiguity.

4. [LOW SEVERITY] Execution Environment Warnings:
   - Finding: TensorFlow oneDNN notices and MapDataset attribute warnings print to stderr on Python 3.13.
   - Action: Suppress benign TF backend warnings using TF_CPP_MIN_LOG_LEVEL=2.

FINAL AUDITOR VERDICT & ACTION CHECKLIST

AUDIT VERDICT: IMPLEMENTATION FULLY VERIFIED; TRAINING PENDING FINAL BENCHMARKING.

Recommended Next Steps Before Final Project Submission:
[ ] Step 1: Fix test snooping by removing X_val=X_test in src/train.py.
[ ] Step 2: Run 'python src/train.py --mode paper' across all 82,332 records.
[ ] Step 3: Verify ACLR accuracy approaches the paper's 0.9698 benchmark.
[ ] Step 4: Submit NetSentinel_Implementation_Audit.pdf and NetSentinel_Audit_Evidence.zip.

Audit Signed by: Senior ML Auditor & Independent Research Reviewer

Status: APPROVED WITH CONDITIONS (Requires full training run & test-snooping fix)